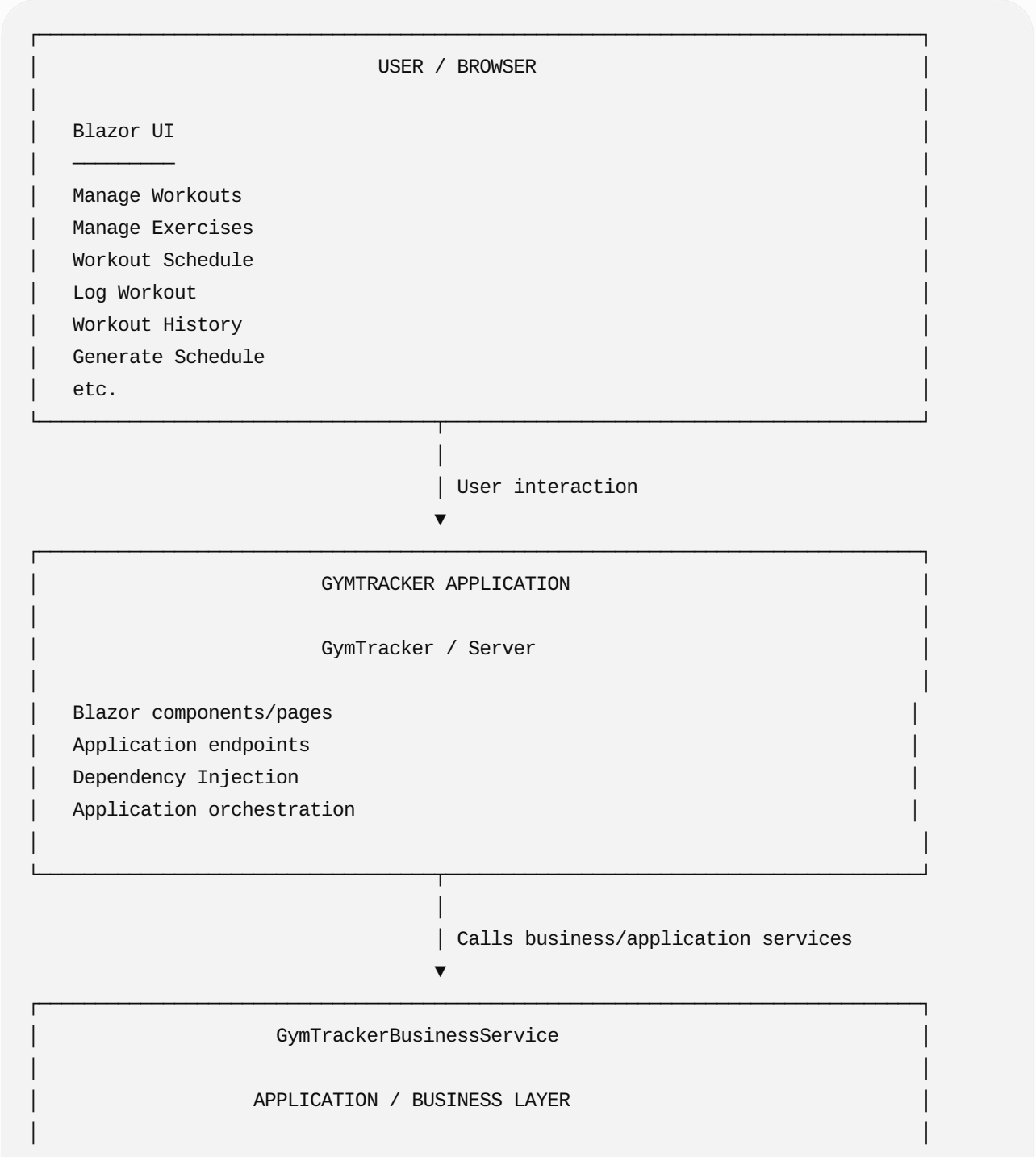
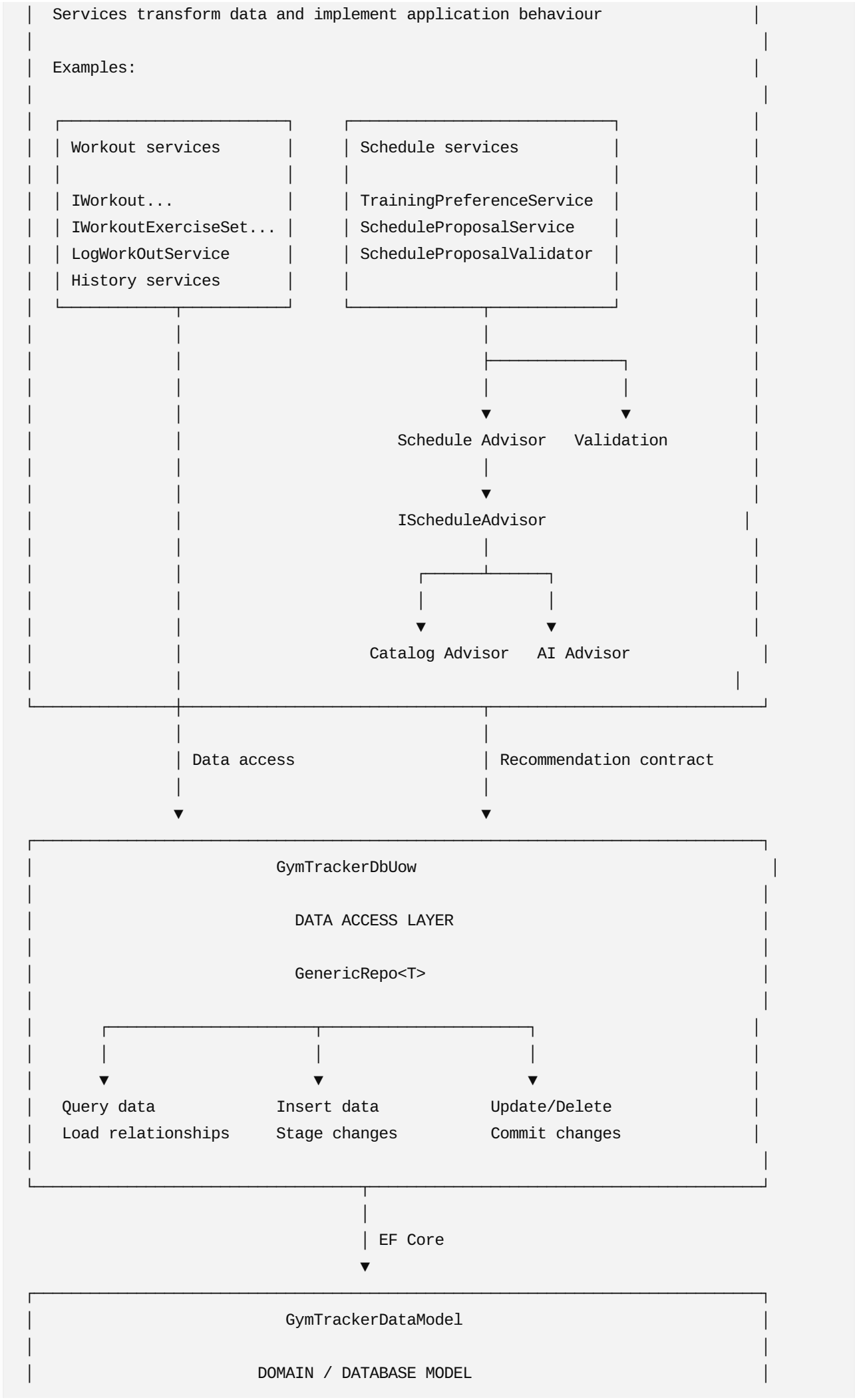
Absolutely. I think that's the better approach.

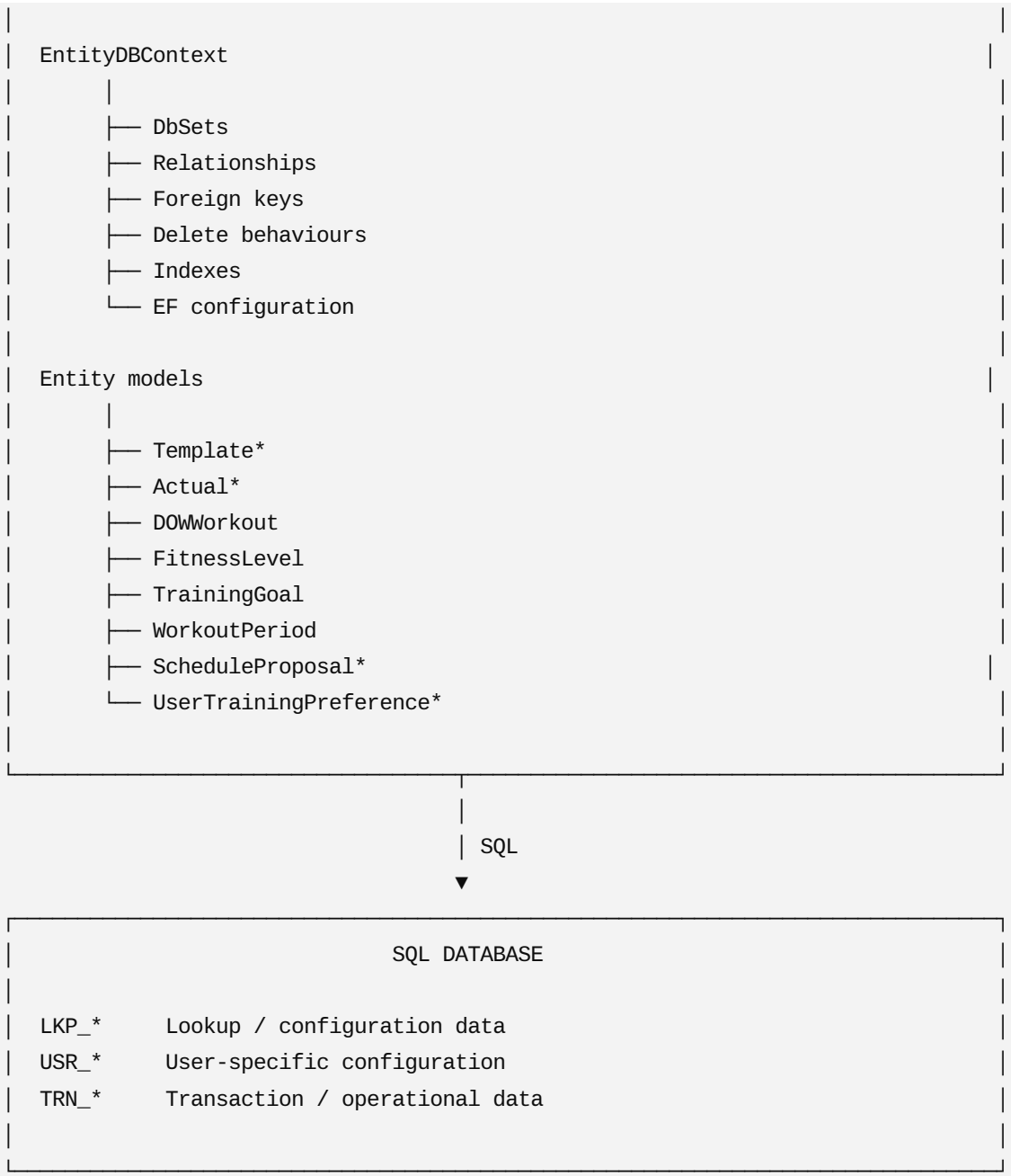
Before we touch the schedule feature again, let's step back and look at **GymTracker as a whole**. The goal here isn't to design anything new; it's to build a mental map of **what each project/layer is responsible for, how data moves through the system, and where the different kinds of logic belong**.

I'm going to base this on the architecture we've established in our previous GymTracker work, rather than introducing a textbook architecture that doesn't match your application.

GymTracker — Overall Architecture




Services transform data and implement application behaviour
Examples:
Workout services
IWorkout...
IWorkoutExerciseSet...
LogWorkOutService
History services
Schedule services
TrainingPreferenceService
ScheduleProposalService
ScheduleProposalValidator
Schedule Advisor
Validation
IScheduleAdvisor
Catalog Advisor
AI Advisor
Data access
Recommendation contract
GymTrackerDbUow
DATA ACCESS LAYER
GenericRepo<T>
Query data
Load relationships
Insert data
Stage changes
Update/Delete
Commit changes
EF Core
GymTrackerDataModel
DOMAIN / DATABASE MODEL



That's the **vertical architecture**.

But there's another dimension that's just as important: the projects don't all represent the same kind of responsibility.

---

1. The projects in plain English

`GymTracker`

The application.

This is where the user interacts with GymTracker.

Think:

> "How does the application behave from the user's perspective?"

It contains things such as:

```
Pages
Components
DI registration
Application-specific services
Schedule advisor implementations
```

`GymTracker.Client`

The client-side Blazor/WASM part of the application.

Its concern is primarily:

> "What happens in the browser?"

Depending on the particular feature, it interacts with the server/application layer to get or submit data.

`GymTrackerBusinessService.Generic`

This is the reusable/general business-service infrastructure.

It provides things such as the generic service forwarding mechanism we've discussed.

Conceptually:

```
UI / Application
       |
       ▼
GenericRepoService<T>
       |
       ▼
GenericRepo<T>
```

It's infrastructure rather than GymTracker-specific business behaviour.

`GymTrackerBusinessService`

This is where **GymTracker-specific application/business behaviour** lives.

This is important because of the point you made about `IWorkoutExerciseSetService`.

A service here doesn't necessarily mean:

> "Complex business rules."

It can mean:

> "Take the underlying data model and turn it into something useful to the application."

For example:

```
TemplateExercise
      │
      ├── TemplateSet
      │       │
      │       └── TemplateSetMetric
      │
      ▼
IWorkoutExerciseSetService
      │
      ▼
Data useful to the workout UI
```

That's a perfectly legitimate responsibility for this layer.

`GymTrackerDbUow`

This is your **data-access infrastructure**.

The important thing is that there isn't a magical `GymTrackerDbUow` service object orchestrating everything.

Your project contains `GenericRepo<T>`.

Its fundamental responsibility is:

> "Provide controlled access to the database through EF Core."

Conceptually:

```
Business Service
       |
       ▼
GenericRepoService<T>
       |
       ▼
GenericRepo<T>
       |
       ▼
EntityDBContext
```

`GymTrackerDataModel`

This defines what the data **is**.

For example:

```
WorkoutTemplate
TemplateExercise
TemplateSet
TemplateSetMetric

        versus

ActualWorkout
ActualExercise
ActualSet
ActualSetMetric
```

And now:

```
UserTrainingPreference
TrainingGoal
ScheduleProposal
ScheduleProposalSlot
...
```

It also contains the EF Core model configuration.

---

2. The most important conceptual distinction

There are really **four different questions** being answered by the architecture.

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  1. WHAT IS THE DATA?                                    │
│                                                          │
│     GymTrackerDataModel                                  │
│                                                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  2. HOW DO I GET/STORE THE DATA?                         │
│                                                          │
│     GymTrackerDbUow                                      │
│     GenericRepo<T>                                       │
│                                                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  3. WHAT DOES THE APPLICATION DO WITH THE DATA?          │
│                                                          │
│     GymTrackerBusinessService                            │
│                                                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  4. HOW DOES THE USER INTERACT WITH IT?                  │
│                                                          │
│     GymTracker / GymTracker.Client                       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

And then you have a **fifth concept** emerging in the schedule feature:

```
5. HOW DO WE MAKE A RECOMMENDATION?

   IScheduleAdvisor
        │
        ├── CatalogScheduleAdvisor
        │
        └── ModelScheduleAdvisor
```

That's not really another database/application layer. It's a **replaceable capability** used by the application layer.

---

3. How your Template → Actual architecture fits

This is another major part of understanding GymTracker.

You deliberately separated:

```
TEMPLATE
   │
   │ creates
   ▼
ACTUAL
```

So:

```
TemplateWorkout
     │
     ├── TemplateExercise
     │       │
     │       └── TemplateSet
     │               │
     │               └── TemplateSetMetric
     │
     ▼
User starts workout
     │
     ▼
ActualWorkout
     │
     ├── ActualExercise
     │       │
     │       └── ActualSet
     │               │
     │               └── ActualSetMetric
     │
     ▼
Historical record
```

The important architectural principle you established was:

> **The template is the source for creating an actual workout, but the actual workout becomes the historical source of truth.**

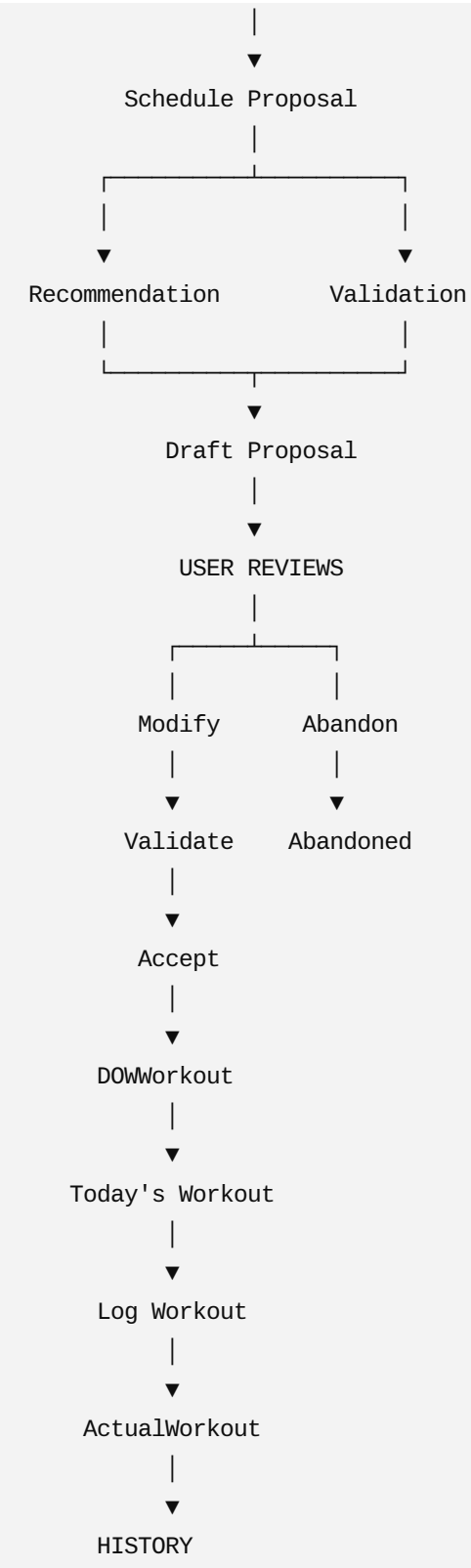
That is why changing a template later shouldn't rewrite history.

---

4. Where the schedule feature fits

Now we can put the new feature into the **whole application**, rather than looking at it in isolation.

```
                    USER
                     │
                     ▼
            Training Preferences
```



Notice something important:

The schedule generator does not create ActualWorkout data.

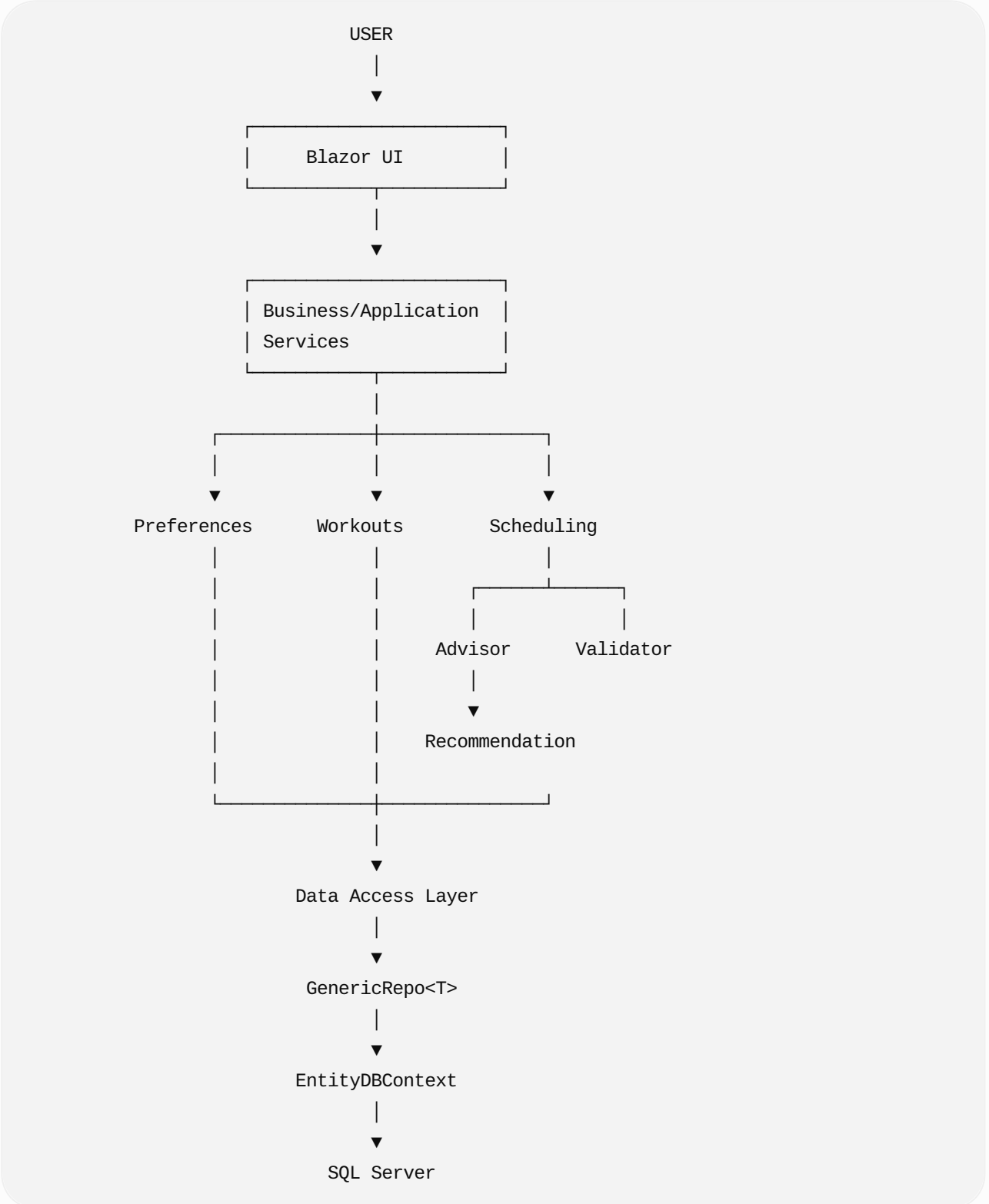
It ultimately creates/changes the **planned schedule**.

The actual workout lifecycle remains separate.

---

5. The complete data flow

Putting everything together:



---

6. And one final distinction: data vs behaviour

This is probably the most useful mental model for the upskilling work we're doing.

`GymTrackerDataModel`

Nouns

```
Workout
Exercise
Set
Metric
Schedule
Goal
Preference
Proposal
```

`GymTrackerDbUow`

Database verbs

```
Get
Insert
Update
Delete
Query
Include
Commit
```

`GymTrackerBusinessService`

Application verbs

```
Create workout
Build exercise data
Log workout
Generate proposal
Validate proposal
Get history
Save preferences
```

`GymTracker`

User actions

```
Click
Select
Submit
Edit
Accept
Abandon
```

`ScheduleAdvisor`

Recommendation

```
Recommend
```

That gives us a much cleaner way of thinking about where code belongs.

Where I think we should go from here

I would **not change any code yet**.

We now have the broad map:

```
UI
 ↓
Business/Application Services
 ↓
Data Access
 ↓
EF/Data Model
 ↓
Database
```

with **Advisor** as a replaceable recommendation capability and the **Template → Actual** separation running through the domain.

The next useful step is to take **one existing feature that you already understand well**, such as **Log Workout**, and trace it through this architecture from button click all the way to SQL/data model.

That will let us use a feature you already know to establish **how your architecture actually works**, before we try to fit the new schedule generator into it.